

CI/CD Report — HW06 (§6, §14)

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **Pipeline:** [.github/workflows/api-tests.yml](#)
- **Trạng thái:** đã chạy 2 lượt thật trên **GitHub Actions** — một XANH, một ĐỎ (§3). Repo bài làm ở trạng thái **PUBLIC**: <https://github.com/DuyITLOR/HW06-API-Testing>

1. Cấu hình pipeline

Mục	Giá trị
Trigger	push vào main khi đổi postman/**, tools/run-newman.sh, tools/ci-gate.mjs, ci/expected-failures.json, workflow · workflow_dispatch (chọn gate_mode)
Runner	ubuntu-latest, timeout 15 phút, concurrency: api-tests (không chạy song song vì tranh cổng 3000 và cùng database.sqlite)
SUT	checkout ttbhanh/eshop-sut ngay trong job → node server.js → chờ tối đa 40s tới khi GET /api/products trả 200
Dữ liệu test	node tools/seed-api-data.mjs (fixture + CSV data-driven)
Chạy test	newman run cho 3 collection, reporter cli,json,htmlextra
Hai cổng	(1) regression suite → ci-gate.mjs --strict, phải 0 đỏ; (2) 3 collection bug-hunting → so với baseline ci/expected-failures.json (29/30/30). gate_mode=strict áp --strict cho cả hai
Bằng chứng	upload artifact newman-<mode>-<run#>: HTML + JSON + sut.log, giữ 30 ngày, if: always()

2. Vì sao cổng không dùng exit code của Newman

Bộ test này cố ý bắt bug thật của SUT, nên số assertion đỏ ở trạng thái bình thường **lớn hơn 0**. Lấy "0 đỏ" làm cổng thì pipeline đỏ vĩnh viễn: mọi lượt đỏ như nhau, và **hồi quy mới không còn phân biệt được với bug cũ** — tức cổng mất hết tác dụng. Baseline chuyển câu hỏi thành "số đỏ có đúng như đã ký nhận không": đỏ tăng = hồi quy mới, đỏ giảm = SUT đã sửa **hoặc test của mình yếu đi**, cả hai đều cần người xem lại.

2.1 Cổng đã được kiểm chứng bằng lượt chạy local (bằng chứng)

```
$ node tools/ci-gate.mjs reports/newman/*.json

== Cổng CI ==
[XANH] api-01-products-search: 29/155 đỏ, khớp baseline (29)
[XANH] api-02-cart-add: 30/81 đỏ, khớp baseline (30)
[XANH] api-03-product-update: 30/93 đỏ, khớp baseline (30)
```

Build XANH.

Cùng dữ liệu đó với `--strict` (chế độ tạo lượt ĐỎ mẫu):

```
$ node tools/ci-gate.mjs reports/newman/*.json --strict
[ĐỎ] api-01-products-search: 29/155 assertion đỏ (chế độ --strict)
...
Build ĐỎ – 3 mục không đạt.
```

Tức hai nhánh của cổng đều đã chạy thật, chỉ còn thiếu **hai lượt trên runner của GitHub**.

3. Hai lượt mẫu (§6 bắt buộc) – **ĐÃ CHẠY**, đúng nghĩa đề đòi

Đề đòi "one whose pipeline run shows all API test cases passing, and another whose pipeline run shows one test case failing". Bộ 136 case không đáp ứng được theo nghĩa chữ — nó cố ý bắt bug thật nên luôn có 89 assertion đỏ. Vì vậy pipeline có **hai bộ, hai cổng, hai vai trò** (xem §2), và hai lượt mẫu dưới đây chạy trên **regression suite**:

Lượt	Committ	Cổng regression	Cổng baseline (136 case)	Build	Link run	Ảnh
Tất cả pass	5a07ebf	216 assertion, 0 đỏ	29/30/30 khớp baseline	success · 39s	runs/32580345226	bug-report/screenshots/ci-xanh.png
Đúng 1 test fail	e388146	1/217 đỏ	(không tới bước này)	failure · 33s	runs/32580407707	bug-report/screenshots/ci-do.png

Trích log lượt XANH:

```
23127178 · HW06 Regression suite
|           assertions |           216 |           0 |
== Cổng CI ==
[XANH] reports/newman/ci-regression.json: 216 assertion, 0 đỏ
Build XANH.
[XANH] ci-api-01-products-search.json: 29/155 đỏ, khớp baseline (29)
[XANH] ci-api-02-cart-add.json: 30/85 đỏ, khớp baseline (30)
[XANH] ci-api-03-product-update.json: 30/93 đỏ, khớp baseline (30)
```

Trích log lượt ĐỎ:

```
1. DEMO cố ý fail – tạo lượt CI đỏ mẫu (§6)
|           assertions |           217 |           1 |
[ĐỎ] reports/newman/ci-regression.json: 1/217 assertion đỏ (chế độ --strict)
Build ĐỎ – 1 mục không đạt.
```

Cách tạo lượt đỏ, và vì sao chọn cách đó. `tools/gen-regression.mjs --break TC-PRODLIST-003` thêm **đúng một** assertion có nhãn `DEMO` cố ý fail rồi commit. Hai cách khác đều tệ hơn: làm hỏng một assertion thật thì **mất một phép kiểm**, còn nói một assertion cho nó fail thì nói sai về phạm vi bộ test. Commit kế tiếp (`4e2f302`) sinh lại collection và gỡ assertion demo, nên lượt đỏ nằm trong lịch sử git nhưng **không** nằm trong bản nộp.

Regression suite được sinh ra, không viết tay. Nó là tập con các case có 0 assertion đỏ ở lượt mới nhất, **giữ nguyên expected**. Nhờ vậy nó không thể "xanh giả" bằng cách nói assertion: mỗi case trong đó là một case của bộ chính. 94 request được giữ, 69 request có assertion đỏ bị loại.

4. Dự đoán trước khi chạy, và kết quả thật

Dự đoán ghi trước khi push: runner dùng `database.sqlite` sạch trong repo SUT, local cũng khởi động lại SUT trước mỗi collection nên cũng sạch → kỳ vọng **khớp baseline 29/30/30**; nếu lệch thì nghi dữ liệu seed trước, không nghi code test.

Kết quả thật: đúng — 29/30/30 trên runner, giống hệt local. Ba điểm đáng ghi lại:

1. **Bản sửa "ghi rồi kiểm chứng bản ghi còn sống" hoạt động trên runner ngay lần thử đầu:** log CI in [OK] user thứ hai `hw06.user2@eshop.com` đã tồn tại và sống sót (lần thử 1). Trên runner, DB là file sạch mới checkout nên không có dữ liệu cũ để phục vụ trong cửa sổ chờ — tức đúng cái điều kiện đã làm lộ ra lỗi #12 ở local lại **không** xảy ra ở CI. Nếu chỉ chạy CI thì lỗi đó đã không bao giờ bị phát hiện.
2. **Số liệu ổn định qua 5 lượt** (3 local + 2 CI) trên hai môi trường khác nhau (macOS arm64 · Ubuntu x86-64, Node v22 · Node 20). Với bài này, con số 29/30/30 là **thuộc tính của SUT**, không phải của máy.
3. **Cổng đúng như thiết kế:** cùng một tập dữ liệu cho ra XANH ở chế độ `baseline` và ĐỎ ở `strict` — tức phần quyết định đỏ/xanh nằm ở **ngưỡng đã ký nhận**, không nằm ở exit code của Newman.

4. Chênh lệch giữa lượt CI và lượt local (điền sau)

DB trên runner là `database.sqlite` sạch trong repo SUT, khác DB local đã seed nhiều lần → số sản phẩm khác, nên các assertion dựa vào **số dòng** phải viết theo kiểu tương đối (so trước/sau) chứ không hard-code. Ghi lại ở đây mọi chỗ phải sửa vì lý do này.